

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 20%

QUESTIONS:5 Total Marks 20 Duration :60 Minutes Annexure A
Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I Harshavardhan. RV (192472163) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Harshavardhan. RV

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	

Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	
--------------	-----	--	--	--------------------------------	------------------------------	--

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1.Real-Time Object Detection Pipeline

A system must:

A. Capture video frames

B. Detect objects using a trained model

C. Draw bounding boxes and labels

D. Display results in real time

Constraints:

- **Must process frames efficiently**
- **Handle multiple objects**

Write detailed pseudocode with modular steps for detection, visualization, and efficient looping.

Ans: Real-Time Object Detection Pipeline

The system captures live video, sends each frame to a trained object-detection model, obtains multiple detected objects, draws bounding boxes and labels, and displays the result continuously.

Overall Pipeline

Camera



Capture Frame



Preprocess Frame



Trained Detection Model



Detect Multiple Objects



Filter Low-Confidence Results



Draw Boxes + Labels



Display Frame



Repeat

Detailed Modular Pseudocode

Module 1: Initialize System

ALGORITHM RealTimeObjectDetection

BEGIN

```
LOAD trained object detection model
LOAD class labels
```

```
INITIALIZE video camera
```

```
SET confidence_threshold = 0.5
SET input_width = 640
SET input_height = 480
```

```
IF camera is not available THEN
    DISPLAY "Camera Error"
    STOP
END IF
```

Module 2: Capture Video Frame

```
WHILE TRUE
```

```
    READ frame from camera
```

```
    IF frame is not successfully captured THEN
        BREAK
    END IF
```

Each iteration captures one frame from the live video stream.

Module 3: Preprocess Frame

```
    RESIZE frame to input_width × input_height
```

```
    CONVERT frame into required model format
```

```
    NORMALIZE pixel values if required
```

```
    PREPARE processed frame as model input
```

Preprocessing ensures that the image has the correct size and format expected by the trained model.

Module 4: Object Detection

```
    SEND processed frame to detection model
```

```
    GET detection results
```

```
    FOR each detected object DO
```

```
        GET class_id
        GET confidence
        GET bounding_box coordinates
```

```
        IF confidence >= confidence_threshold THEN
            STORE object information
        END IF
```

```
    END FOR
```

Because the loop processes **every detected object**, multiple objects can be handled in the same frame.

For example:

Frame

—	Person	0.92
—	Car	0.87
—	Bicycle	0.81
—	Dog	0.76

Module 5: Visualization

FOR each valid detected object DO

 GET x, y, width, height

 DRAW rectangle around object

 GET object class name

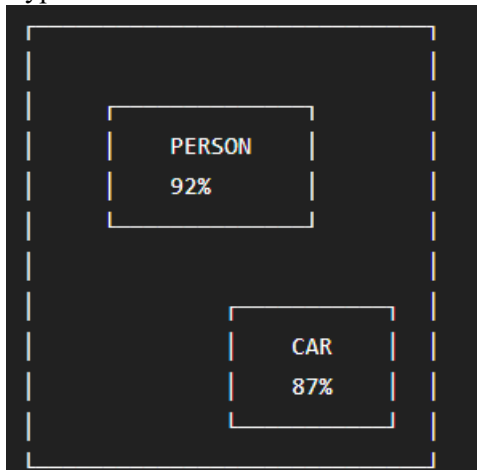
 CREATE label:

 class_name + confidence

 DRAW label above bounding box

END FOR

Typical visualization:



Module 6: Display Result

DISPLAY annotated frame in window

IF user presses 'Q' THEN

 BREAK

END IF

END WHILE

Module 7: Release Resources

RELEASE video camera

CLOSE all display windows

END

Complete Pseudocode

ALGORITHM RealTimeObjectDetection

BEGIN

 LOAD trained detection model

 LOAD object class labels

 OPEN camera

 SET confidence_threshold = 0.5

 SET frame_width = 640

 SET frame_height = 480

 IF camera is not available THEN

 PRINT "Unable to access camera"

 EXIT

 END IF

 WHILE TRUE

 frame ← READ_FRAME(camera)

 IF frame is empty THEN

 BREAK

 END IF

 frame ← RESIZE(frame, 640, 480)

 input ← PREPROCESS(frame)

 detections ← MODEL_DETECT(input)

 FOR each detection IN detections DO

 confidence ← detection.confidence

 IF confidence >= 0.5 THEN

 class_id ← detection.class_id

 box ← detection.bounding_box

 DRAW_RECTANGLE(frame, box)

 class_name ← labels[class_id]

 label ← class_name +

 " " +

 confidence

 DRAW_TEXT(frame, label, box.position)

 END IF

 END FOR

 SHOW(frame)

 key ← GET_KEY()

 IF key = 'Q' THEN

 BREAK

 END IF

END WHILE

```
RELEASE(camera)
CLOSE_WINDOWS()
```

END

Efficient Loop Design

To improve real-time performance:

Live Frame



Resize



Fast Preprocessing



Single Model Inference



Process All Detections



Draw Results



Display

Additional optimization can include:

- Use a smaller input resolution.
- Use a lightweight trained model.
- Ignore detections below the confidence threshold.
- Process only the required region of interest when appropriate.
- Avoid unnecessary image copies.
- Use frame skipping if the model is too slow.

Constraint Justification

Requirement	Solution
Capture video	Continuous camera loop
Detect objects	Trained detection model
Multiple objects	Loop through all detections
Bounding boxes	DRAW_RECTANGLE()
Labels	DRAW_TEXT()
Real-time display	Process and display each frame
Efficient processing	Resize + lightweight model
False detections	Confidence threshold
Continuous operation	WHILE TRUE loop
Safe termination	Keyboard exit + resource release

Conclusion

The proposed pipeline separates the system into **initialization, frame capture, preprocessing, detection, visualization, and cleanup modules**. A single frame can contain multiple objects because all valid detections are processed in a loop. Bounding boxes and labels are drawn directly on the frame before displaying it. Using a lightweight model, reduced resolution, confidence filtering, and efficient frame processing helps achieve **real-time object detection with minimal delay**.

2. Image Difference Detection System

A system must:

A. Read two images

B. Compare them

C. Highlight differences

Constraints:

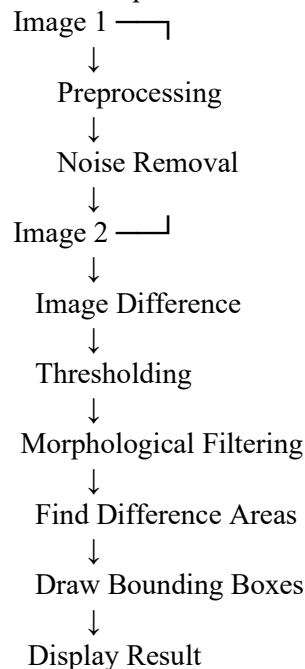
- Handle noise
- Ensure accurate difference detection

Write pseudocode including preprocessing, comparison logic, and visualization.

Ans: Image Difference Detection System

The system reads two images, preprocesses them to reduce noise, compares corresponding pixels, identifies significant differences, and highlights those differences visually.

Overall Pipeline



Detailed Pseudocode

Module 1: Read Images

ALGORITHM ImageDifferenceDetection

BEGIN

 READ Image1

 READ Image2

 IF Image1 or Image2 cannot be read THEN

 DISPLAY "Image loading error"

 STOP

 END IF

Module 2: Preprocessing

Both images must have the same size before comparison.

 RESIZE Image1 to required dimensions

 RESIZE Image2 to same dimensions

 CONVERT Image1 to grayscale

 CONVERT Image2 to grayscale

APPLY Gaussian Blur to Image1

APPLY Gaussian Blur to Image2

Gaussian blur reduces small noise that could otherwise appear as false differences.

Image1 $\leftarrow$ GaussianBlur(Image1)

Image2 $\leftarrow$ GaussianBlur(Image2)

Module 3: Image Comparison

Calculate the absolute difference between corresponding pixels.

Difference $\leftarrow$ ABSOLUTE_DIFFERENCE(Image1, Image2)

Conceptually:

Image 1 Image 2

↓

↓

Pixel A

Pixel B

\

/

Difference

If the pixel values are similar, the difference will be small. If they are different, the difference will be large.

Module 4: Thresholding

Convert the difference image into a binary mask.

DifferenceMask $\leftarrow$ THRESHOLD(
 Difference,
 threshold_value
)

Small Difference $\rightarrow$ Background

Large Difference $\rightarrow$ Difference

The threshold prevents tiny variations caused by noise from being detected as actual changes.

Module 5: Morphological Filtering

Remove small unwanted regions.

CREATE morphological kernel

DifferenceMask $\leftarrow$ MORPHOLOGICAL_OPENING(
 DifferenceMask,
 kernel
)

DifferenceMask $\leftarrow$ MORPHOLOGICAL_CLOSING(
 DifferenceMask,
 kernel
)

Opening removes small noise, while closing fills small gaps inside difference regions.

Module 6: Find Difference Regions

FIND contours in DifferenceMask

FOR each contour DO

 area $\leftarrow$ CALCULATE_CONTOUR_AREA(contour)

 IF area > minimum_area THEN

 bounding_box $\leftarrow$ FIND_BOUNDING_RECTANGLE(contour)

 STORE bounding_box

 END IF

END FOR

The minimum-area condition prevents tiny noise regions from being treated as differences.

Module 7: Visualization

Draw rectangles around the detected differences.

FOR each bounding_box DO

 DRAW rectangle on Image1

 DRAW rectangle on Image2 or output image

 ADD label "Difference"

END FOR

DISPLAY DifferenceMask

DISPLAY highlighted output image

Complete Pseudocode

ALGORITHM ImageDifferenceDetection

BEGIN

 Image1 ← READ("image1.jpg")

 Image2 ← READ("image2.jpg")

 IF Image1 = NULL OR Image2 = NULL THEN

 PRINT "Error: Images not found"

 STOP

 END IF

 Image1 ← RESIZE(Image1, required_size)

 Image2 ← RESIZE(Image2, required_size)

 Gray1 ← CONVERT_TO_GRAYSCALE(Image1)

 Gray2 ← CONVERT_TO_GRAYSCALE(Image2)

 Gray1 ← GAUSSIAN_BLUR(Gray1)

 Gray2 ← GAUSSIAN_BLUR(Gray2)

 Difference ← ABSOLUTE_DIFFERENCE(Gray1, Gray2)

 DifferenceMask ← THRESHOLD(

 Difference,
 threshold_value

)

 Kernel ← CREATE_KERNEL()

 DifferenceMask ← MORPH_OPEN(

 DifferenceMask,
 Kernel

)

 DifferenceMask ← MORPH_CLOSE(

 DifferenceMask,
 Kernel

)

```
Contours ← FIND_CONTOURS(DifferenceMask)
```

```
FOR each Contour in Contours DO
```

```
    Area ← CONTOUR_AREA(Contour)
```

```
    IF Area > minimum_area THEN
```

```
        Box ← BOUNDING_RECTANGLE(Contour)
```

```
        DRAW_RECTANGLE(
```

```
            Image1,
```

```
            Box
```

```
        )
```

```
        DRAW_TEXT(
```

```
            Image1,
```

```
            "Difference",
```

```
            Box.position
```

```
        )
```

```
    END IF
```

```
END FOR
```

```
DISPLAY "Original Image 1", Image1
```

```
DISPLAY "Difference Mask", DifferenceMask
```

```
WAIT_FOR_KEY()
```

END

Noise Handling

Noise in Images



Gaussian Blur



Pixel Difference



Thresholding



Morphological Opening



Minimum Area Filtering



Accurate Differences

These steps prevent minor camera noise, compression artifacts, and tiny pixel variations from being incorrectly classified as meaningful changes.

Constraint Justification

Requirement	Solution
Read two images	READ()
Compare images	Absolute pixel difference

Noise handling	Gaussian blur
Accurate detection	Threshold + minimum-area filtering
Remove small noise	Morphological opening
Fill gaps	Morphological closing
Locate changes	Contour detection
Highlight differences	Bounding rectangles and labels
Visualization	Difference mask + highlighted image

Conclusion

The proposed image difference system first resizes and converts both images to grayscale, then applies Gaussian filtering to reduce noise. The images are compared using absolute pixel differences, followed by thresholding to identify meaningful changes. Morphological operations and minimum-area filtering remove false differences. Finally, contours and bounding boxes are used to highlight the exact regions that have changed, providing accurate and easy-to-understand difference detection.

3. Video Frame Extraction System

A system must:

A. Read video input

B. Extract frames at fixed intervals

C. Save selected frames

Constraints:

- Efficient storage
- Avoid redundant frames

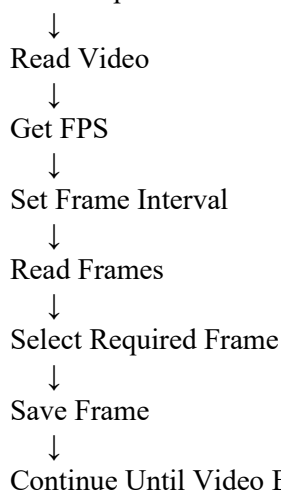
Write structured pseudocode for frame selection and saving logic.

Ans: **Video Frame Extraction System**

The system reads a video, extracts frames at fixed time or frame intervals, and saves only the required frames. This reduces storage usage and avoids saving unnecessary consecutive frames.

Overall Pipeline

Video Input



Structured Pseudocode

Module 1: Initialize Video

ALGORITHM VideoFrameExtraction

BEGIN

 OPEN video file

```
IF video cannot be opened THEN
    DISPLAY "Video Error"
    STOP
END IF
```

```
GET video FPS
GET total number of frames
```

```
SET interval = required frame interval
```

```
frame_count ← 0
saved_count ← 0
```

Module 2: Read Frames

```
WHILE video has frames
```

```
    READ current frame
```

```
    IF frame is not available THEN
        BREAK
    END IF
```

```
    frame_count ← frame_count + 1
```

Module 3: Select Frames at Fixed Intervals

For example, if the video is 30 FPS and we want one frame every 2 seconds:

```
Interval = FPS × 2
          = 30 × 2
          = 60 frames
```

Therefore, save every 60th frame.

```
IF frame_count MOD interval = 0 THEN
```

```
    SAVE current frame
```

```
    saved_count ← saved_count + 1
```

```
END IF
```

Module 4: Avoid Redundant Frames

To avoid saving frames that are almost identical, an optional difference check can be used.

```
IF selected frame is reached THEN
```

```
    CONVERT current frame to grayscale
```

```
    IF previous saved frame exists THEN
```

```
        difference ← COMPARE(
            current frame,
            previous saved frame
        )
```

```

    IF difference < similarity_threshold THEN
        SKIP current frame
    ELSE
        SAVE current frame
        previous_frame ← current frame
    END IF

```

```

ELSE
    SAVE current frame
    previous_frame ← current frame
END IF

```

```

END IF

```

This prevents nearly identical frames from occupying storage.

Complete Pseudocode

ALGORITHM VideoFrameExtraction

```

BEGIN

```

```

    // ----- Video Initialization -----
    OPEN video

```

```

    IF video is not available THEN
        PRINT "Unable to open video"
        STOP
    END IF

```

```

    FPS ← GET_FPS(video)

```

```

    SET seconds_interval ← 2

```

```

    frame_interval ← FPS × seconds_interval

```

```

    frame_count ← 0

```

```

    saved_count ← 0

```

```

    previous_frame ← NULL

```

```

    // ----- Frame Processing -----
    WHILE TRUE

```

```

        frame ← READ_FRAME(video)

```

```

        IF frame = NULL THEN
            BREAK
        END IF

```

```

frame_count ← frame_count + 1

// ----- Fixed Interval Selection -----
IF frame_count MOD frame_interval = 0 THEN

    gray_current ← CONVERT_TO_GRAYSCALE(frame)

    // ----- Redundancy Check -----
    IF previous_frame ≠ NULL THEN

        difference ← CALCULATE_FRAME_DIFFERENCE(
            gray_current,
            previous_frame
        )

        IF difference < similarity_threshold THEN
            CONTINUE
        END IF

    END IF

    // ----- Save Frame -----
    filename ←
        "frame_" + saved_count + ".jpg"

    SAVE_FRAME(frame, filename)

    previous_frame ← gray_current

    saved_count ← saved_count + 1

END IF

END WHILE

// ----- Cleanup -----
CLOSE video

PRINT "Frames saved:", saved_count

```

END

Example

For a **30 FPS video**:

Required interval = 2 seconds

Frame 0 → Save
Frame 30 → Check
Frame 60 → Save
Frame 90 → Check
Frame 120 → Save

...

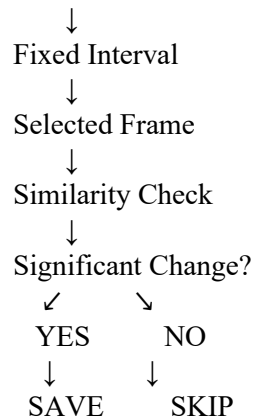
If the system saves one frame every 2 seconds, a 10-minute video produces approximately:

$600 \text{ seconds} \div 2 = 300$ selected frames

The redundancy check can further reduce this number when consecutive selected frames are very similar.

Efficient Storage Strategy

Continuous Video



Constraint Justification

Requirement	Solution
Read video	Video capture/read loop
Fixed interval	$\text{FPS} \times \text{time interval}$
Save frames	SAVE FRAME()
Efficient storage	Save only selected frames
Avoid redundant frames	Frame similarity check
Large videos	Sequential processing
Memory efficiency	Process one frame at a time
Easy organization	Sequential frame filenames

Conclusion

The proposed system reads the video **one frame at a time**, calculates the required frame interval using the video FPS, and saves only selected frames. A **frame similarity check** can additionally remove redundant frames that contain almost no visual change. Since frames are processed sequentially rather than loading the entire video into memory, the system provides **efficient storage and memory usage** while extracting useful frames at fixed intervals.

4. Background Subtraction Pipeline

A surveillance system must:

A. Read video stream

B. Separate foreground from background

C. Highlight moving objects

Constraints:

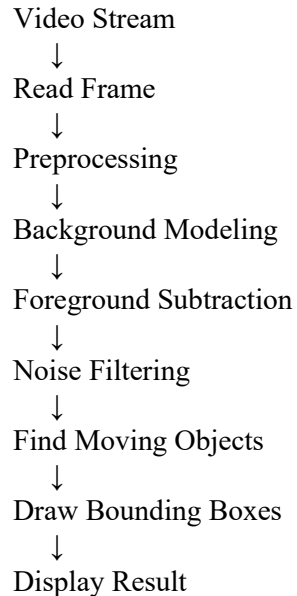
- Handle dynamic background
- Reduce noise

Write pseudocode including background modeling, subtraction, filtering, and display.

Ans: **Background Subtraction Pipeline**

A background subtraction system separates **moving foreground objects** such as people and vehicles from the stationary or slowly changing background in a video.

Overall Pipeline



Structured Pseudocode

Module 1: Initialize Video and Background Model

ALGORITHM BackgroundSubtraction

BEGIN

 OPEN video stream

 IF video cannot be opened THEN

 DISPLAY "Video Error"

 STOP

 END IF

 CREATE adaptive background model

 SET noise_threshold

 SET minimum_object_area

An adaptive model such as **MOG2** is suitable because it can update the background when the scene changes.

Module 2: Read Video Frames

 WHILE TRUE

 READ frame from video

 IF frame is not available THEN

 BREAK

 END IF

Module 3: Preprocessing

RESIZE frame if required

CONVERT frame to grayscale

APPLY Gaussian Blur

gray_frame ← GaussianBlur(gray_frame)

Blurring reduces small camera noise before subtraction.

Module 4: Background Modeling and Subtraction

foreground_mask ←

BACKGROUND_SUBTRACTOR(gray_frame)

The adaptive background model continuously learns the scene.

Current Frame



Compare with Background Model



Foreground Mask



Moving Objects

Module 5: Noise Filtering

CREATE morphological kernel

foreground_mask ←

MORPH_OPEN(
foreground_mask,
kernel
)

foreground_mask ←

MORPH_CLOSE(
foreground_mask,
kernel
)

- **Opening** removes small noise.
- **Closing** fills small holes in detected objects.

Module 6: Detect Moving Objects

contours ← FIND_CONTOURS(foreground_mask)

FOR each contour DO

area ← CONTOUR_AREA(contour)

IF area > minimum_object_area THEN

box ← BOUNDING_RECTANGLE(contour)

DRAW_RECTANGLE(

```

        frame,
        box
    )

    DRAW_TEXT(
        frame,
        "Moving Object",
        box.position
    )

END IF

```

```

END FOR

```

Small regions are ignored so that noise is not displayed as moving objects.

Complete Pseudocode

ALGORITHM BackgroundSubtraction

```

BEGIN

```

```

    // ----- Initialization -----
    video ← OPEN_VIDEO_STREAM()

```

```

    IF video = NULL THEN
        PRINT "Unable to open video"
        STOP
    END IF

```

```

    background_model ←
        CREATE_ADAPTIVE_BACKGROUND_MODEL()

```

```

    SET minimum_object_area ← 500

```

```

    kernel ← CREATE_MORPHOLOGICAL_KERNEL()

```

```

    // ----- Main Processing Loop -----
    WHILE TRUE

```

```

        frame ← READ_FRAME(video)

```

```

        IF frame = NULL THEN
            BREAK
        END IF

```

```

        // ----- Preprocessing -----
        frame ← RESIZE(frame)

```

```

gray ← CONVERT_TO_GRAYSCALE(frame)

gray ← GAUSSIAN_BLUR(gray)

// ----- Background Subtraction -----
foreground ←
    background_model.APPLY(gray)

// ----- Noise Filtering -----
foreground ←
    MORPH_OPEN(foreground, kernel)

foreground ←
    MORPH_CLOSE(foreground, kernel)

// ----- Object Detection -----
contours ←
    FIND_CONTOURS(foreground)

FOR each contour IN contours DO

    area ← CONTOUR_AREA(contour)

    IF area > minimum_object_area THEN

        box ← BOUNDING_RECTANGLE(contour)

        DRAW_RECTANGLE(
            frame,
            box
        )

        DRAW_TEXT(
            frame,
            "Moving Object",
            box.position
        )

    END IF

END FOR

// ----- Display -----
SHOW("Original + Detection", frame)

```

```
SHOW("Foreground Mask", foreground)
```

```
// ----- Exit -----
```

```
IF USER_PRESSES('Q') THEN
```

```
    BREAK
```

```
END IF
```

```
END WHILE
```

```
// ----- Cleanup -----
```

```
RELEASE(video)
```

```
CLOSE_ALL_WINDOWS()
```

```
END
```

Handling Dynamic Backgrounds

Dynamic backgrounds can include:

- Moving trees
- Rain
- Water
- Shadows
- Changing illumination

An **adaptive background model** such as **MOG2** continuously updates its representation of the background.

Dynamic Background



Adaptive Model



Continuous Update



Better Foreground Separation

This is better than using a single fixed background image.

Noise Reduction

Camera Noise



Gaussian Blur



Background Subtraction



Morphological Opening



Morphological Closing



Minimum Area Filtering



Clean Foreground

These steps prevent tiny noise regions from being incorrectly identified as moving objects.

Constraint Justification

Requirement	Solution
Read video	Continuous video-reading loop
Separate foreground	Adaptive background subtraction
Dynamic background	MOG2/adaptive background model
Reduce noise	Gaussian blur
Remove small regions	Morphological opening
Fill object gaps	Morphological closing
Ignore tiny changes	Minimum-area threshold
Highlight objects	Bounding rectangles
Display results	Original frame + foreground mask
Continuous operation	Real-time processing loop

Conclusion

The proposed background subtraction pipeline uses an **adaptive background model, preprocessing, foreground subtraction, morphological filtering, contour detection, and visualization**. The adaptive model handles changing backgrounds, while Gaussian blur and morphological operations reduce noise. Minimum-area filtering prevents small unwanted regions from being detected. Finally, bounding boxes highlight moving objects, making the system suitable for **real-time surveillance applications**.

5. Edge Detection Pipeline for Real-Time Video

A system must:

A. Capture video frames

B. Apply edge detection

C. Display results

Constraints:

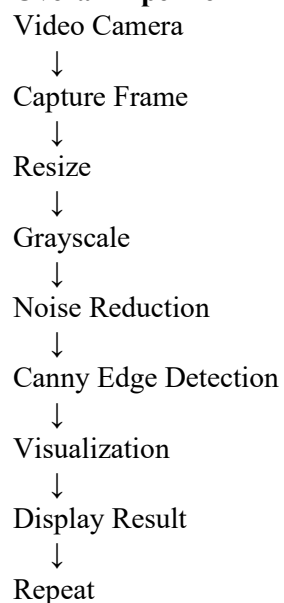
- **Real-time processing**
- **Maintain clarity of edges**

Write pseudocode with preprocessing, edge detection, and visualization modules.

Ans: Edge Detection Pipeline for Real-Time Video

The system captures live video frames, preprocesses them to reduce noise, applies an edge detection algorithm such as **Canny**, and displays the detected edges continuously.

Overall Pipeline



Structured Pseudocode

Module 1: Initialize Video

ALGORITHM RealTimeEdgeDetection

BEGIN

 OPEN video camera

 IF camera cannot be opened THEN

 DISPLAY "Camera Error"

 STOP

 END IF

 SET frame_width = 640

 SET frame_height = 480

 SET lower_threshold = 50

 SET upper_threshold = 150

Module 2: Capture Frame

 WHILE TRUE

 READ frame from camera

 IF frame is not available THEN

 BREAK

 END IF

The loop continuously processes incoming frames.

Module 3: Preprocessing

 RESIZE frame to 640×480

 gray $\leftarrow$ CONVERT_TO_GRAYSCALE(frame)

 gray $\leftarrow$ GAUSSIAN_BLUR(
 gray,
 kernel_size = 5
)

Grayscale reduces computation, while Gaussian blur removes small noise that could create unwanted edges.

Module 4: Edge Detection

Use the **Canny edge detector**:

 edges $\leftarrow$ CANNY(
 gray,
 lower_threshold,
 upper_threshold
)

Canny detects strong intensity changes and produces clear edge boundaries.

Input Frame



```

Grayscale
↓
Gaussian Blur
↓
Canny
↓
Edge Image

```

Module 5: Visualization

The edge image can be displayed separately or overlaid on the original frame.

```
DISPLAY "Original Video", frame
```

```
DISPLAY "Detected Edges", edges
```

For an overlay:

```
edge_overlay ← CONVERT_TO_COLOR(edges)
```

```
DISPLAY edge_overlay
```

Complete Pseudocode

ALGORITHM RealTimeEdgeDetection

BEGIN

```
// ----- Initialization -----
```

```
video ← OPEN_CAMERA()
```

```
IF video = NULL THEN
```

```
    PRINT "Unable to access camera"
```

```
    STOP
```

```
END IF
```

```
width ← 640
```

```
height ← 480
```

```
lower_threshold ← 50
```

```
upper_threshold ← 150
```

```
// ----- Main Processing Loop -----
```

```
WHILE TRUE
```

```
    // 1. Capture
```

```
    frame ← READ_FRAME(video)
```

```
    IF frame = NULL THEN
```

```
        BREAK
```

```
    END IF
```

```
    // 2. Preprocessing
```

```

frame ← RESIZE(frame, width, height)

gray ← CONVERT_TO_GRAYSCALE(frame)

gray ← GAUSSIAN_BLUR(
    gray,
    (5,5)
)

// 3. Edge Detection
edges ← CANNY(
    gray,
    lower_threshold,
    upper_threshold
)

// 4. Visualization
SHOW("Original Frame", frame)

SHOW("Edge Detection", edges)

// 5. Exit
IF USER_PRESSES('Q') THEN
    BREAK
END IF

END WHILE

// ----- Cleanup -----
RELEASE(video)
CLOSE_ALL_WINDOWS()

END

```

Maintaining Edge Clarity
The main factors affecting edge quality are **noise and threshold selection**.

```

Noisy Frame
↓
Gaussian Blur
↓
Clean Grayscale Image
↓
Canny Detection
↓
Clear Edges

```


The two Canny thresholds control which intensity changes are considered meaningful edges. They should be adjusted according to the camera and environment.

Real-Time Optimization

To maintain real-time performance:

- Resize frames to **640 × 480**.
- Convert to grayscale before edge detection.
- Use a small Gaussian kernel.
- Avoid unnecessary image copies.
- Use optimized OpenCV functions.
- If processing is still slow, process every second frame.

Constraint Justification

Requirement	Solution
Capture video	Continuous camera loop
Edge detection	Canny algorithm
Noise reduction	Gaussian blur
Clear edges	Proper Canny thresholds
Real-time processing	Reduced resolution + optimized operations
Display results	OpenCV display functions
Continuous operation	Frame-by-frame processing

Conclusion

The proposed pipeline uses **frame capture, resizing, grayscale conversion, Gaussian filtering, Canny edge detection, and visualization**. Gaussian filtering reduces noise, while Canny produces clear and well-defined edges. Processing at a suitable resolution and using lightweight OpenCV operations ensures that the system can operate **continuously in real time with clear edge results**.